

NotHarshhaa / DevOps-Projects

CodeIssues2Pull requests18ProjectsSecurity and qualityInsights

masterDevOps-Projects / DevOps-Project-01 /Go to fileAdd file

NotHarshhaaDevOps-Project-01: Update Java Login App and Infrastructure Modulesefa9ae9 · 3 months agoHistory

| Name           | Last commit message                                                 | Last commit date |
|----------------|---------------------------------------------------------------------|------------------|
| ..             |                                                                     |                  |
| Java-Login-App | DevOps-Project-01: Update Java Login App and Infrastructure Modules | 3 months ago     |
| infrastructure | DevOps-Project-01: Update Java Login App and Infrastructure Modules | 3 months ago     |
| README.md      | DevOps: README: Update blog link across all DevOps projects         | 4 months ago     |

README.md

## Deploy Java Application on AWS 3-Tier Architecture

The diagram illustrates a 3-tier AWS architecture within a single VPC. It is organized into two Availability Zones: us-east-1a and us-east-1b. Each zone contains a Public Subnet and a Private Subnet. The Public Subnets (Public Subnet 1a and Public Subnet 1b) are connected to the Internet via Internet Gateways. The Private Subnets (Private Subnet 1a and Private Subnet 1b) are connected to the Public Subnets via VPC Peering. The architecture is secured by Security Groups. The diagram also shows the VPC, Subnets, and the Internet Gateway.

### Table of Contents

- 1. [Project Overview](#)
- 2. [Architecture Overview](#)
- 3. [Pre-Requisites](#)
- 4. [Infrastructure Setup](#)
  - [VPC and Networking](#)
  - [Security Configuration](#)
  - [Database Layer](#)
- 5. [Application Setup](#)
  - [Build Environment](#)
  - [Application Deployment](#)
  - [Load Balancing and Auto Scaling](#)
- 6. [Monitoring and Maintenance](#)
- 7. [Security Best Practices](#)
- 8. [Troubleshooting Guide](#)
- 9. [Contributing](#)

AWS

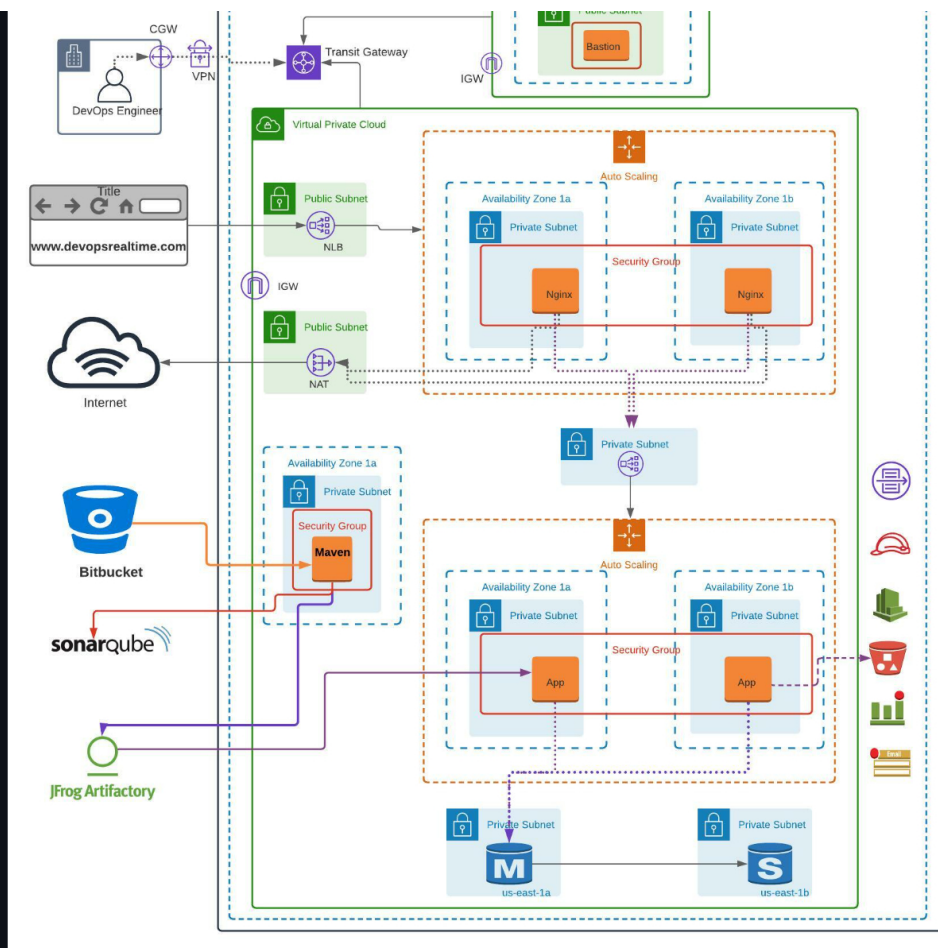
AWS Cloud

Region

Virtual Private Cloud

Availability Zone

Dublin Subnet



## Project Overview

### Introduction

This project demonstrates the deployment of a production-grade Java web application using AWS's robust 3-tier architecture. The implementation follows cloud-native best practices, ensuring high availability, scalability, and security across all application tiers.

### Key Features

- **High Availability:** Multi-AZ deployment with automated failover
- **Auto Scaling:** Dynamic resource allocation based on demand
- **Security:** Defense-in-depth approach with multiple security layers
- **Monitoring:** Comprehensive logging and monitoring setup
- **Cost Optimization:** Efficient resource utilization and management

### Architecture Overview

#### Infrastructure Components

1. **Presentation Tier (Frontend)**
  - Nginx web servers in Auto Scaling Group
  - Public-facing Network Load Balancer
  - CloudFront Distribution for static content
2. **Application Tier (Backend)**
  - Apache Tomcat servers in Auto Scaling Group
  - Internal Network Load Balancer
  - Session management with Amazon ElastiCache
3. **Data Tier**
  - Amazon RDS MySQL in Multi-AZ configuration
  - Automated backups and point-in-time recovery
  - Read replicas for read-heavy workloads

#### Network Architecture

- **VPC Design**
  - Two separate VPCs (192.168.0.0/16 and 172.32.0.0/16)

- Public and private subnets across multiple AZs
- Transit Gateway for inter-VPC communication

## Pre-Requisites

### Required Accounts and Tools

#### 1. AWS Account Setup

- Create an [AWS Free Tier Account](#)
- Install AWS CLI v2

```
# For Linux
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
unzip awscliv2.zip
sudo ./aws/install

# For macOS
brew install awscli

# Configure AWS CLI
aws configure
```

#### 2. Development Tools

- Git: Version control system

```
# For Linux
sudo apt-get update
sudo apt-get install git

# For macOS
brew install git
```

#### 3. CI/CD Integration

- SonarCloud Account
  - Sign up at [SonarCloud](#)
  - Generate authentication token
  - Configure project settings:

```
# Add to pom.xml
<properties>
  <sonar.projectKey>your_project_key</sonar.projectKey>
  <sonar.organization>your_organization</sonar.organization>
  <sonar.host.url>https://sonarcloud.io</sonar.host.url>
</properties>
```

- JFrog Artifactory
  - Create account on [JFrog Cloud](#)
  - Set up Maven repository
  - Configure authentication:

```
<!-- settings.xml -->
<servers>
  <server>
    <id>jfrog-artifactory</id>
    <username>${env.JFROG_USERNAME}</username>
    <password>${env.JFROG_PASSWORD}</password>
  </server>
</servers>
```

## Infrastructure Setup

### VPC and Networking

#### 1. VPC Creation

```
# Create primary VPC
aws ec2 create-vpc \
  --cidr-block 192.168.0.0/16 \
  --tag-specifications "ResourceType=vpc,Tags=[{Key=Name,Value=PrimaryVPC}]" \
  --region us-east-1

# Create secondary VPC
aws ec2 create-vpc \
  --cidr-block 172.32.0.0/16 \
  --tag-specifications "ResourceType=vpc,Tags=[{Key=Name,Value=SecondaryVPC}]" \
  --region us-east-1
```

#### 2. Subnet Configuration

```
# Create public subnet
aws ec2 create-subnet \
  --vpc-id vpc-xxx \
```

```
aws ec2 create-subnet \
  --cidr-block 192.168.1.0/24 \
  --availability-zone us-east-1a \
  --tag-specifications "ResourceType=subnet,Tags=[{Key=Name,Value=PublicSubnet1}]"

# Create private subnet
aws ec2 create-subnet \
  --vpc-id vpc-xxx \
  --cidr-block 192.168.2.0/24 \
  --availability-zone us-east-1b \
  --tag-specifications "ResourceType=subnet,Tags=[{Key=Name,Value=PrivateSubnet1}]"
```

### 3. Gateway Setup

```
# Create and attach Internet Gateway
aws ec2 create-internet-gateway
aws ec2 attach-internet-gateway --vpc-id vpc-xxx --internet-gateway-id igw-xxx

# Create NAT Gateway
aws ec2 create-nat-gateway \
  --subnet-id subnet-xxx \
  --allocation-id eipalloc-xxx \
  --tag-specifications "ResourceType=natgateway,Tags=[{Key=Name,Value=PrimaryNATGateway}]"
```

## Security Configuration

### 1. Security Groups

```
# Create frontend security group
aws ec2 create-security-group \
  --group-name FrontendSG \
  --description "Security group for frontend servers" \
  --vpc-id vpc-xxx

# Allow inbound HTTP/HTTPS
aws ec2 authorize-security-group-ingress \
  --group-id sg-xxx \
  --protocol tcp \
  --port 80 \
  --cidr 0.0.0.0/0

aws ec2 authorize-security-group-ingress \
  --group-id sg-xxx \
  --protocol tcp \
  --port 443 \
  --cidr 0.0.0.0/0
```

### 2. IAM Roles and Policies

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:PutObject"
      ],
      "Resource": "arn:aws:s3:::your-bucket/*"
    }
  ]
}
```

## Database Layer

### 1. RDS Instance Creation

```
aws rds create-db-instance \
  --db-instance-identifier prod-mysql \
  --db-instance-class db.t3.medium \
  --engine mysql \
  --master-username admin \
  --master-user-password "YourSecurePassword" \
  --allocated-storage 20 \
  --multi-az \
  --vpc-security-group-ids sg-xxx \
  --db-subnet-group-name your-db-subnet-group
```

### 2. Database Initialization

```
-- Connect to database
mysql -h your-rds-endpoint -u admin -p

-- Create application database
CREATE DATABASE javaapp;
USE javaapp;

-- Create users table
CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  username VARCHAR(50) NOT NULL UNIQUE,
```

```
password VARCHAR(255) NOT NULL,  
email VARCHAR(100) NOT NULL UNIQUE,  
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
  
-- Create necessary indexes  
CREATE INDEX idx_username ON users(username);  
CREATE INDEX idx_email ON users(email);
```

# Application Setup

## Build Environment

### 1. Maven Configuration

```
<!-- pom.xml -->  
<project>  
  <properties>  
    <java.version>11</java.version>  
    <spring.version>2.5.12</spring.version>  
  </properties>  
  
  <dependencies>  
    <!-- Add your dependencies here -->  
  </dependencies>  
  
  <build>  
    <plugins>  
      <plugin>  
        <groupId>org.springframework.boot</groupId>  
        <artifactId>spring-boot-maven-plugin</artifactId>  
      </plugin>  
    </plugins>  
  </build>  
</project>
```

### 2. Build Process

```
# Clean and build project  
mvn clean package -DskipTests  
  
# Run tests  
mvn test  
  
# Deploy to JFrog  
mvn deploy
```

## Application Deployment

### 1. Tomcat Configuration

```
# Create tomcat.service  
sudo tee /etc/systemd/system/tomcat.service << EOF  
[Unit]  
Description=Apache Tomcat Web Application Container  
After=network.target  
  
[Service]  
Type=forking  
Environment=JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64  
Environment=CATALINA_PID=/opt/tomcat/temp/tomcat.pid  
Environment=CATALINA_HOME=/opt/tomcat  
Environment=CATALINA_BASE=/opt/tomcat  
Environment='CATALINA_OPTS=-Xms512M -Xmx1024M -server -XX:+UseParallelGC'  
Environment='JAVA_OPTS=-Djava.awt.headless=true -Djava.security.egd=file:/dev/./urandom'  
  
ExecStart=/opt/tomcat/bin/startup.sh  
ExecStop=/opt/tomcat/bin/shutdown.sh  
  
User=tomcat  
Group=tomcat  
UMask=0007  
RestartSec=10  
Restart=always  
  
[Install]  
WantedBy=multi-user.target  
EOF
```

### 2. Nginx Configuration

```
# /etc/nginx/conf.d/app.conf  
upstream backend {  
    server internal-nlb-xxx.elb.amazonaws.com:8080;  
}  
  
server {  
    listen 80;  
    server_name example.com;  
  
    location / {
```

```
    proxy_pass http://backend;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
}

location /static/ {
    proxy_pass https://your-cloudfront-distribution.cloudfront.net;
}
}
```

## Load Balancing and Auto Scaling

### 1. Launch Template Configuration

```
aws ec2 create-launch-template \
  --launch-template-name WebServerTemplate \
  --version-description WebServerVersion1 \
  --launch-template-data '{
    "ImageId": "ami-xxx",
    "InstanceType": "t3.micro",
    "SecurityGroupIds": ["sg-xxx"],
    "UserData": "IyEvYmluL2Jhc2gkCm9ScW5zdGFsbCB0Z2lueApzdWVlH11bSBpbmN0Ym91IG9naW54IC15Cg=="
  }'
```

### 2. Auto Scaling Group

```
aws autoscaling create-auto-scaling-group \
  --auto-scaling-group-name WebServerASG \
  --launch-template LaunchTemplateName=WebServerTemplate,Version='>Latest' \
  --min-size 2 \
  --max-size 6 \
  --desired-capacity 2 \
  --vpc-zone-identifier "subnet-xxx,subnet-yyy" \
  --target-group-arns "arn:aws:elasticloadbalancing:region:account-id:targetgroup/your-target-group/xxx" \
  --health-check-type ELB \
  --health-check-grace-period 300
```

## Monitoring and Maintenance

### CloudWatch Setup

#### 1. Metrics Configuration

```
# Create custom metric for memory usage
cat << EOF > /opt/aws/scripts/memory-metrics.sh
#!/bin/bash
MEMORY_USAGE=$(free | grep Mem | awk '{print $3/$2 * 100.0}')
aws cloudwatch put-metric-data \
  --metric-name MemoryUsage \
  --namespace CustomMetrics \
  --value $MEMORY_USAGE \
  --dimensions InstanceId=$(curl -s http://169.254.169.254/latest/meta-data/instance-id)
EOF

# Add to crontab
echo "* * * * * /opt/aws/scripts/memory-metrics.sh" | crontab -
```

#### 2. Log Management

```
# Configure CloudWatch agent
cat << EOF > /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.json
{
  "agent": {
    "metrics_collection_interval": 60,
    "run_as_user": "root"
  },
  "logs": {
    "logs_collected": {
      "files": {
        "collect_list": [
          {
            "file_path": "/opt/tomcat/logs/catalina.out",
            "log_group_name": "/aws/tomcat/application",
            "log_stream_name": "{instance_id}",
            "timezone": "UTC"
          }
        ]
      }
    }
  },
  "metrics": {
    "metrics_collected": {
      "mem": {
        "measurement": [
          "mem_used_percent"
        ]
      },
      "swap": {
        "measurement": [
          "swap_used_percent"
        ]
      }
    }
  }
}
```

```
    }
  }
}
EOF
```

# Security Best Practices

## 1. Network Security

- Implement network ACLs
- Use security groups effectively
- Enable VPC Flow Logs
- Configure AWS WAF

## 2. Application Security

- Regular security patches
- Implement AWS Shield
- Use AWS Secrets Manager
- Enable AWS GuardDuty

## 3. Data Security

- Enable encryption at rest
- Use SSL/TLS for data in transit
- Regular security audits
- Implement backup strategies

# Troubleshooting Guide

## Common Issues and Solutions

### 1. Connection Issues

```
# Check connectivity
telnet database-endpoint 3306

# Verify security group rules
aws ec2 describe-security-groups --group-ids sg-xxx

# Test load balancer health
aws elbv2 describe-target-health --target-group-arn arn:aws:elasticloadbalancing:region:account-id:targetgroup/your-target-group/xxx
```

### 2. Performance Issues

```
# Check CPU usage
top -bn1

# Monitor memory usage
free -m

# Check disk usage
df -h

# Monitor Tomcat threads
ps -eLf | grep java | wc -l
```

# Contributing

## How to Contribute

1. Fork the repository
2. Create a feature branch
3. Commit your changes
4. Push to the branch
5. Create a Pull Request

## Development Setup

```
# Clone repository
git clone https://github.com/yourusername/your-repo.git

# Install dependencies
mvn install

# Run tests
mvn test
```

---

## ✂ Author & Community

This project is maintained by [Harshhaa](#) 🧑. Your feedback and contributions are welcome!

📧 Connect with me:

- GitHub: [@NotHarshhaa](#)
- Blog: [ProDevOpsGuy](#)
- Telegram Community: [Join Here](#)
- LinkedIn: [Harshhaa Vardhan Reddy](#)

---

## ★ Support the Project

If you found this project helpful, please consider:

- Starring ★ the repository
- Sharing it with your network
- Contributing to its improvement

## 📢 Stay Connected



# prodevopsguy

🔗 DEVOPS & CLOUD TECH COMMUNITY 🏠

👤 @NotHarshhaa

---

### 📌 Important

This documentation is continuously evolving. For the latest updates, please check the repository regularly.